MODPKICA Java 复刻版

开 发 助 手

源码结构分析 · 运行流程解析 · 常量变量函数用法速查

技术栈：Java + BouncyCastle 1.85（纯 javac 离线编译）

版本：v1.0 | 作者：@魔帝本尊 | 2026-09

目 录

1. 项目概述
2. 总体架构与模块划分
3. 源码结构分析（21 个类）
4. 运行流程解析
5. 常量与变量速查
6. 公共函数用法速查
7. 配置文件与数据格式
8. WebAdmin 面板与 API
9. 附录：构建与命令行速查

1. 项目概述

MODPKICA 是一套完整的 PKI（公钥基础设施） 教学与工具软件，原版为 Go 语言实现。本项目使用 Java + BouncyCastle 1.85 离线复刻其全部功能，共 7 个主程序 + 1 个浏览器管理面板，覆盖 X.509 证书签发、CRL 吊销列表、OCSP 在线状态、RFC3161 / Authenticode 时间戳、证书验证等场景。

设计目标：纯 javac 编译、零网络依赖、BC 1.85 全家桶本地化（BC185/ 目录），便于离线开发与学习 x.509 ASN.1 编码。

项目	说明
复刻对象	Go 版 MODPKICA（cmd/main、cmd/admin、cmd/makeroot、cmd/makecert、cmd/rootgetcrl、cmd/certverify、cmd/auto）
开发语言	Java 17+（使用 java.base 的 com.sun.net.httpserver 实现 HTTP 服务，无第三方 Web 框架）
加密库	BouncyCastle 1.85（bcprov / bcpkix / bcutil / bcmail）
构建	build.cmd（Windows）/ build.sh（Linux），javac -encoding UTF-8 -d out
运行	java -Dfile.encoding=UTF-8 -cp "out;BC185*" modpkica.<主程序>
数据目录	MODPKICA_BASE（环境变量或系统属性）指向 PKI 根，默认取当前工作目录
证书体系	多根模型：全部证书按序列号命名（序列号.crt / 序列号.key），无单默认根
密钥算法	RSA / ECC（ECDSA）/ SM2（国密），签名摘要自动适配（SM2 → SM3）

七个主程序与对应类：

主程序	类	职责
MAIN	MAIN.java	网络服务：OCSP、CRL 下载、时间戳（RFC3161 + Authenticode）、静态网页、内嵌 WebAdmin
ADMIN	Admin.java	管理控制台：init / makeroot / signcert / revoke / signCRL / initOCSP / initTIMESTAMP / verifyCERT / list
MAKEROOT	MakeRoot.java	签发根证书（类型 R）
MAKECERT	MakeCert.java	签发下级证书（中间 CA 类型 C / 终端类型 E）
ROOTGETCRL	RootGetCrl.java	遍历 R/C 证书生成全部 CRL
CERTVERIFY	CertVerify.java	证书验证：字段解析、签名验证、证书链、CRL/OCSP 吊销状态、哈希
AUTO	Auto.java	自动化：init / revoke / newcert
WEBADMIN	WebAdmin.java	浏览器管理面板（管理端口 8080，13 页面 16 API）

2. 总体架构与模块划分

项目按 主程序层 → 服务/业务层 → 工具层 → 数据层 组织，21 个类全部位于 src/modpkica/ 包内，无子包依赖，结构扁平。

```
主程序层（main 入口，7+1）
  MAIN / Admin / MakeRoot / MakeCert / RootGetCrl / CertVerify / Auto / WebAdmin
        |
服务/业务层（被主程序调用）
  TimestampUtil  OcspUtil  CrlUtil  CertUtil  Sm2Util  X509JsonConverter
        |
工具/数据层
  CertsDB（CERTS.txt）  Config（CONFIG.txt）  MLog（logs.log）
  PemIO（PEM 读写）  Bc（Provider 注册）  AppPaths（工作目录）  AppEnv（程序清单）
```

关键设计约定：

① 多根模型：不再有 root.crt / root.key 单根概念，全部证书（含根）按序列号落盘；OCSP 证书按序列号_ocsp.crt/.key、TSA 证书按 类型_哈希算法.crt 命名。

② 自适应算法：证书签发/OCSP 响应/时间戳签名按密钥类型自动选择摘要与签名算法（RSA/ECC → SHA1~SHA512；SM2 → SM3）。

③ 登记表驱动：CERTS.txt 是证书状态唯一权威来源，MAIN 在每次 OCSP/时间戳处理前重新加载（db.reload()），外部修改即时生效。

④ 全量日志：所有操作通过 MLog 追加写 logs.log，含时间、时间戳、操作、命令、结果。

3. 源码结构分析（21 个类）

MAIN —— 主程序 1：常驻网络服务

src/modpkica/MAIN.java

关键字段/常量：baseDir、cfg(Config)、db(CertsDB)、adminPort

关键方法：

- ☐ main(args) 启动流程：定位工作目录 → 加载 Config/CertsDB → 刷新 CRL → 创建 HttpServer → 注册路由 → 启动 WebAdmin
- ☐ handleOcsp(ex) /OCSP /ocsp：解析请求 → db.reload() → 查登记表 → 状态判定 → 加载 OCSP 签名证书 → 构造响应
- ☐ handleRfc3161(ex) /rfc3161：RFC3161 文档签名时间戳（SHA1/SHA256 自适应）
- ☐ handleAuthenticode(ex,digitype,tsaGroup) /timestamp /timestamp/sha1：Authenticode 时间戳
- ☐ handleFile(ex) /：WebPublic 静态文件（CRT/CRL 下载）
- ☐ pickTSACert(hashName) 自适应根 TSA 选证书：sha1→rsa_sha1/ecc_sha1 随机；sha256→rsa_sha256/ecc_sha256；sm3→sm2_sm3
- ☐ extractAuthenticodeContent(data) 提取被签名内容（0x04 0x81/82/83 分割，对齐 Go 版）

Admin —— 主程序 2：管理控制台

src/modpkica/Admin.java

关键字段/常量：VERSION = "MODPKICA Java 复刻版 v1.0 (BC 1.85 离线)"

关键方法：

- ☐ doInit(baseDir,dn,bit,hash,algo) 初始化环境（清空证书目录 + 重建登记表）
- ☐ doMakeRoot(baseDir,dn,bit,hash,algo) 签发根证书
- ☐ doList(baseDir) 列出全部登记证书
- ☐ doSignCert(baseDir,dn,bit,hash,type,validYears,sanType,sanContent,issuerSerial,algo) 签发下级证书
- ☐ doRevoke(baseDir,serial,reason) 吊销证书
- ☐ doSignCrl(baseDir) 刷新全部 CRL
- ☐ doInitOcsp(baseDir,algo) 为全部 R/C 证书签发 OCSP 签名证书
- ☐ doInitTimestamp(baseDir,hash) 签发全部时间戳证书（rsa/ecc/sm2 × sha1/sha256/sm3）
- ☐ doVerifyCert(baseDir,target) 证书验证

MakeRoot —— 主程序 3：签发根证书

src/modpkica/MakeRoot.java

关键字段/常量：—

关键方法：

- ☐ main(args) / run(baseDir,args)：参数 证书DN 密钥位数 哈希算法 密钥算法；产物 ROOT/序列号.crt+.key，同步 WebPublic/CRT，登记 CERTS.txt（R）

MakeCert —— 主程序 4：签发下级证书

src/modpkica/MakeCert.java

关键字段/常量：—

关键方法：

- ☐ main(args) / run(baseDir,args)：按颁发者序列号从 ROOT/CA 目录加载密钥对签发；支持 RSA/ECC/SM2、EKU/KeyUsage/SAN/策略/有效期

RootGetCrl —— 主程序 5：生成全部 CRL

src/modpkica/RootGetCrl.java

关键字段/常量：—

关键方法：

- ☐ run(baseDir)：遍历 CERTS.txt 全部 R/C 证书 → 每个颁发者生成独立 CRL（WebPublic/CRL/序列号.crl），哈希算法沿用颁发者证书

CertVerify —— 主程序 6：证书验证

src/modpkica/CertVerify.java

关键字段/常量：—

关键方法：

- ☐ main(args) / run(filename) 无点号参数按序列号从 WebPublic/CRT 读取（自动小写）
- ☐ verifySignature(cert,pub) SM2 用 SM2Signer 验签，其余用 JCA Signature
- ☐ fetchIssuer(cert) 从 AIA calssuers 拉取颁发者证书
- ☐ checkCrl(cert) / checkOcsp(cert) 吊销状态检查
- ☐ printSctList(value) SCT 列表解析打印

Auto —— 主程序 7：自动化

src/modpkica/Auto.java

关键字段/常量：—

关键方法：

- ☐ init 初始化环境；revoke 序列号 原因；newcert 序列号 类型 状态 原因 时间 颁发者ID 算法

WebAdmin —— 浏览器管理面板

src/modpkica/WebAdmin.java

关键字段/常量：port() 管理端口（系统属性 modpkica.admin.port，默认 8080）

关键方法：

- ☐ main(args) 单独运行面板（GUI only）
- ☐ handleApi(ex) 16 个 /api/* 路由分发
- ☐ runAction(ex,invoker) 执行管理动作：解析 JSON → 调用 doXxx → 写日志（含提交数据全文）→ 返回输出

TimestampUtil —— 时间戳服务核心

src/modpkica/TimestampUtil.java

关键字段/常量：TSA_POLICY_OID = 1.3.6.1.4.1.4146.1.95

关键方法：

- ☐ parseRfc3161Request(der) 解析 RFC3161 请求
- ☐ createRfc3161Response(requestDer,tsaCert,chain,signer,digestCalc) 生成 RFC3161 响应（BC TSP）
- ☐ createRfc3161ResponseStandard(...) RSA/ECC 便捷方法
- ☐ createAuthenticodeTimestamp(content,cert,key,digitype) PKCS#7 分离签名：eContent=内容 + signedAttrs 含 id-data(1.7.1)=内容属性
- ☐ SM3DigestCalculator 自定义 SM3 摘要计算器（SM2 TSA）

OcspUtil —— OCSP 协议编解码

src/modpkica/OcspUtil.java

关键字段/常量：OID_NONCE=1.3.6.1.5.5.7.48.1.2；GOOD=0 / REVOKED=1 / UNKNOWN=2

关键方法：

- ☐ parseRequest(der) 解析 OCSP 请求（序列号/哈希算法/Nonce）
- ☐ createResponse(keyType,responderCert,responderKey,issuerCert,serial,status,reason,revokedAt,digestOid,nonce)

生成签名响应

- [ ] unauthorizedResponse() 5 字节无签名响应（未知序列号）

CrlUtil —— CRL 生成工具

src/modpkica/CrlUtil.java

关键字段/常量：—

关键方法：

- [ ] generateCrl(caCert,caKey,revokedList,hashName) 生成 CRL 字节
- [ ] derToPem(der) DER→PEM

CertUtil —— 证书构建工具

src/modpkica/CertUtil.java

关键字段/常量：—

关键方法：

- [ ] getSubjectKeyIdentifier(pubKey) 计算 SKI
- [ ] readJcaPrivateKey(path) 读取 PKCS#1/#8/SEC1 私钥
- [ ] 证书扩展组装（SKI/AKI/EKU/KeyUsage/SAN/AIA/CRLDP/策略）

Sm2Util —— 国密 SM2/SM3

src/modpkica/Sm2Util.java

关键字段/常量：OID_SM3_WITH_SM2=1.2.156.10197.1.501；OID_SM3=1.2.156.10197.1.401；OID_SM2_PUBLIC_KEY=1.2.156.10197.1.301

关键方法：

- [ ] getSm2UidBytes() 默认 UID（1234567812345678）
- [ ] sm3(data) SM3 摘要
- [ ] computeZa(pub,uid) ZA 值计算
- [ ] sm2Sign(priv,tbs) SM2 签名
- [ ] sm2Verify(pub,tbs,sig) SM2 验签
- [ ] toEcPublicParams(pub) 转为 BC ECPublicKeyParameters

CertsDB —— CERTS.txt 登记表

src/modpkica/CertsDB.java

关键字段/常量：CertEntry{serial,type,status,reason,time,issuerId,algo}

关键方法：

- [ ] load(baseDir) 加载登记表
- [ ] reload() 重新从磁盘加载（外部修改后调用）
- [ ] findBySerial(serial) 按序列号查找（忽略大小写）
- [ ] findIssuerIdBySerial(serial) 查颁发者序列号
- [ ] getAll() 全部条目

Config —— CONFIG.txt 配置

src/modpkica/Config.java

关键字段/常量：键：WEBPORT/OCSP/CRL/CRT/CPS/CPS_NONICE

关键方法：

- [ ] load 加载（WEBPORT 缺省 80）
- [ ] getCpsWithCid(certId) 替换 {CID} 占位符
- [ ] getCpsNonice() 用户通告文本（\n 还原换行）

MLog —— 操作日志

src/modpkica/MLog.java

关键字段/常量：logs.log（只追加，记录间两个换行）

关键方法：

- ☐ append(baseDir,operation,result) 写日志
- ☐ append(baseDir,operation,command,result) 带命令写日志
- ☐ appendErr(baseDir,operation,command,error) 写错误

PemIO —— PEM 读写

src/modpkica/PemIO.java

关键字段/常量：—

关键方法：

- ☐ writeCert(path,cert) 写证书 PEM
- ☐ writePrivateKey(path,key,keyType) 写私钥 PEM（按类型）
- ☐ detectKeyType(pemPath) 探测密钥类型（RSA/EC/SM2）
- ☐ readCert(path) 读证书

AppPaths / AppEnv / Bc / X509JsonConverter —— 基础工具

src/modpkica/

关键字段/常量：—

关键方法：

- ☐ AppPaths.baseDir() 工作目录（环境变量/系统属性 MODPKICA_BASE，缺省当前目录）
- ☐ AppEnv.getExeNames() 主程序清单
- ☐ Bc.init() 注册 BC Provider
- ☐ X509JsonConverter.certToJson(pemBytes) 证书→JSON（证书库详情）

4. 运行流程解析

4.1 MAIN 启动流程

```
MAIN.main()
  ├── AppPaths.baseDir() 定位工作目录（MODPKICA_BASE）
  ├── new Config(baseDir) 加载 PKI/CONFIG.txt（WEBPORT 默认 80）
  ├── new CertsDB().load(baseDir) 加载 PKI/CERTS.txt
  ├── RootGetCrl.run() 启动即刷新全部 CRL（遍历 R/C 颁发者）
  ├── HttpServer.create(端口=WEBPORT)
  |     ├── /OCSP, /ocsp    → handleOcsp
  |     ├── /timestamp      → handleAuthenticode(digitype=2, sha256)
  |     ├── /timestamp/sha1→ handleAuthenticode(digitype=1, sha1)
  |     ├── /rfc3161        → handleRfc3161
  |     └── /               → handleFile（WebPublic 静态：CRT/CRL 下载）
  └── 启动内嵌 WebAdmin（管理端口 8080/系统属性 modpkica.admin.port）
```

4.2 HTTP 路由表

路径	处理器	说明
/OCSP、/ocsp	handleOcsp	OCSP 在线状态（POST, application/ocsp-request）
/timestamp	handleAuthenticode	Authenticode 时间戳 SHA256（微软代码签名）
/timestamp/sha1	handleAuthenticode	Authenticode 时间戳 SHA1
/rfc3161	handleRfc3161	RFC3161 文档签名时间戳（SHA1/SHA256 自适应）
/CRT/{CID}.crt	handleFile	证书下载（WebPublic/CRT）
/CRL/{CID}.crl	handleFile	CRL 下载（WebPublic/CRL）
/CRL/{CID}.der.crl	handleFile	CRL DER 格式下载
/	handleFile	WebPublic 静态文件
管理端口 /*	WebAdmin	面板页面与 /api/*（默认 8080）

注：/timestamp 收到 Content-Type=application/timestamp-query 的请求时自动转交 handleRfc3161。

4.3 OCSP 请求处理流程

```
客户端 POST /OCSP（openssl ocsp / Wosign / 浏览器）
  ├── 读请求体 → 写 PKI/OCSP/ocsp.req（调试留档）
  ├── OcspUtil.parseRequest(req)
  |     ├── 序列号 serialHex（BigInteger.toString(16)）
  |     ├── 哈希算法 OID
  |     └── Nonce（可选）
  ├── db.reload()  ← 每次请求重载 CERTS.txt（外部修改即时生效）
  ├── db.findBySerial(serialHex)
  |     ├── 未命中 → 返回 5 字节无签名响应（unauthorized, 无 responseBytes）
  |     └── 命中 → 状态判定：R=已吊销(含原因/时间) / V=正常 / 其它=未知
  ├── 加载 OCSP 签名证书：PKI/OCSP/颁发者序列号_ocsp.crt/.key（多根模型）
  ├── 加载真正颁发者证书（ROOT/CA 目录）计算 CertID issuer 哈希
  ├── 摘要算法：请求含摘要 → 用请求摘要（sha1/256/384/512）；无 → 默认 sha1；SM2 → SM3
  └── OcspUtil.createResponse(...) 构造签名响应返回
```

4.4 RFC3161 时间戳流程（/rfc3161）

```
客户端 POST（Content-Type: application/timestamp-query, DER 二进制）
  ├── parseRfc3161Request：哈希算法 OID / messageImprint / nonce / certReq
  ├── 自适应根 TSA 选证书：SHA1→rsa_sha1|ecc_sha1 随机；SHA256→rsa_sha256|ecc_sha256；SM3→sm2_sm3
  ├── 通过 TSA 证书序列号查登记表（db.reload()）→ 加载颁发者证书进链
  ├── 签名器：RSA/ECC → JCA；SM2 → SM3withSM2（摘要固定 SM3，回传 hashAlgorithm 也按 SM3）
  └── createRfc3161Response：BC TimeStampTokenGenerator 生成响应 DER
        （SerialNumber=当前时间秒数；GenTime=now UTC；Policy=1.3.6.1.4.1.4146.1.95）
```

4.5 Authenticode 时间戳流程（/timestamp）

```
客户端 POST（微软 Authenticode：base64 文本体，实测含 \r\n 与末尾 \0）
  ├── 请求体 UTF-8 → 只保留 base64 字符集（replaceAll("[^A-Za-z0-9+/=]",""))
  ├── Base64.getDecoder() 解码 → DER
```

```
├── extractAuthenticodeContent：找 0x04 0x81/82/83 分割点取被签名内容（对齐 Go 版）
├── 自适应根 TSA 选证书 + 加载颁发者进链
└── createAuthenticodeTimestamp（PKCS#7 分离签名，响应=base64 PEM 正文）：
    ├── eContent = 完整提取内容（不是摘要）
    ├── SignerInfo.signedAttrs = 标准属性(contentType=id-data, signingTime, messageDigest)
    │    + 额外 id-data(1.2.840.113549.1.7.1) 属性，值=完整内容（Wosign 校验必需）
    └── 签名算法：digitype 1/2/3/4 → SHA1/256/384/512 with RSA|ECDSA；SM2 → SM3withSM2
```

4.6 CRL 生成流程（RootGetCrl / signCRL）

```
遍历 CERTS.txt 全部 R/C 证书（颁发者）
├── 从 ROOT|CA/序列号.crt/.key 加载颁发者密钥对
├── 收集该颁发者下状态=R 的证书（序列号/吊销时间/原因）
├── CrlUtil.generateCrl：BC X509v2CRLBuilder 构造（ThisUpdate/NextUpdate/吊销项）
├── 哈希算法沿用颁发者证书签名算法（如 RSA+sha256 / RSA+sha512）
└── 写 WebPublic/CRL/序列号.crl（+ .der.crl）
```

4.7 证书签发流程

```
MAKEROOT：DN 位数 哈希 算法
├── 生成密钥对 → 构造 X509v3 根证书（CA=TRUE, KeyCertSign+CRLSign, SKI/AKI, 策略含 CPS+UserNotice）
├── 写 ROOT/序列号.crt + ROOT/序列号.key
└── 登记 CERTS.txt（类型 R, 状态 V）

MAKECERT：DN 位数 哈希 类型 年限 SAN类型 SAN内容 颁发者序列号 算法
├── 按颁发者序列号查登记表 → 类型 R→ROOT/ C→CA 目录加载颁发者密钥对
├── 类型 C：CA=TRUE；类型 E：终端（按策略识别 SSL/代码签名/文档签名/OCSP/TSA）
├── 扩展：SKI/AKI、EKU、KeyUsage、SAN、AIA(OCSP+calssuers)、CRLDP、证书策略(CPS+UserNotice)
└── 产物按类型落盘（R→ROOT、C→CA、E→CERT+KEY），全部同步 WebPublic/CRT/序列号.crt，登记 CERTS.txt
```

4.8 吊销流程

```
ADMIN revoke 序列号 原因（或 WebAdmin /api/revoke、Auto revoke）
├── 查登记表 → 更新状态=R、原因编号、吊销时间
├── 写回 CERTS.txt（备份旧文件 .old.*）
└── 后续：signCRL 生成新 CRL；OCSP 请求时按登记表返回 revoked(原因/时间)
```

4.9 日志流程（MLog）

```
MLog.append(baseDir, operation, command, result)
├── 每条记录：时间(yyyy/MM/dd HH:mm:ss)、时间戳(毫秒)、操作、命令、结果
├── 追加写 PKI/logs.log（只追加不覆盖），记录间以两个换行分隔
└── 覆盖场景：Admin 所有操作、WEB 请求（含提交数据 JSON 全文）、
    OCSP/CRL/时间戳请求与响应、签名/摘要/验签过程
```

5. 常量与变量速查

5.1 国密 OID（Sm2Util）

常量	值	含义
OID_SM3_WITH_SM2	1.2.156.10197.1.501	SM3 摘要 + SM2 签名算法
OID_SM3	1.2.156.10197.1.401	SM3 摘要算法
OID_SM2_PUBLIC_KEY	1.2.156.10197.1.301	SM2 公钥算法

5.2 OCSP 状态与扩展（OcspUtil）

常量	值	含义
OID_NONCE	1.3.6.1.5.5.7.48.1.2	Nonce 扩展 OID
GOOD	0	证书正常
REVOKED	1	证书已吊销
UNKNOWN	2	证书状态未知

5.3 证书策略 OID（前端识别 + 签发）

OID	含义
2.23.140.1.1	EV（扩展验证，SSL）
2.23.140.1.2.1	DV（域名验证，SSL）
2.23.140.1.2.2	OV（组织验证，SSL）
2.23.140.1.2.3	IV（个体验证，SSL）
2.23.140.1.3	EV 代码签名
2.23.140.1.4.1	普通代码签名
2.23.140.1.4.2	TSA 证书
1.3.6.1.5.5.7.3.36	通用文档签名
1.3.6.1.4.1.311.10.3.12	微软文档签名
1.3.6.1.4.1.4146.1.95	本系统 TSA 策略（TimestampUtil.TSA_POLICY_OID）

5.4 常用 EKU OID（签发面板 35 项节选）

OID	用途
1.3.6.1.5.5.7.3.1	服务器身份验证
1.3.6.1.5.5.7.3.2	客户端身份验证
1.3.6.1.5.5.7.3.3	代码签名
1.3.6.1.5.5.7.3.4	安全电子邮件
1.3.6.1.5.5.7.3.8	时间戳
1.3.6.1.5.5.7.3.9	OCSP 签名
1.3.6.1.4.1.311.61.1.1	内核模式代码签名
1.3.6.1.4.1.311.10.3.6	Windows 系统组件验证
1.3.6.1.4.1.311.10.3.5	Windows 硬件驱动程序验证
1.3.6.1.4.1.311.10.3.12	文档签名(微软)
1.3.6.1.5.5.7.3.36	通用文档签名
1.3.6.1.4.1.311.10.3.19	已吊销列表签名者

5.5 CERTS.txt 状态/类型/原因

字段	取值	含义
类型	R / C / E	根证书 / 中间证书 / 终端证书
状态	V / R / N	正常 / 已吊销 / 未知
原因	0-9	吊销原因编号（0=未指定 … 5=停止运营 … 9=其他）
时间	yyyy/MM/dd-HH:mm:ss	签发时间或吊销时间

5.6 目录路径约定

路径（相对 MODPKICA_BASE）	内容
PKI/ROOT/	根证书 序列号.crt + 序列号.key
PKI/CA/	中间证书 序列号.crt + 序列号.key
PKI/CERT/	终端证书 PEM 序列号.crt
PKI/KEY/	终端证书私钥 序列号.key
PKI/OCSP/	OCSP 证书 序列号_ocsp.crt/.key
PKI/TIMSTAMP/	TSA 证书 类型_哈希.crt/.key
PKI/WebPublic/CRT/	全部证书对外下载 序列号.crt
PKI/WebPublic/CRL/	全部 CRL 序列号.crl / .der.crl
PKI/CERTS.txt	证书登记表
PKI/CONFIG.txt	系统配置
PKI/logs.log	操作日志

6. 公共函数用法速查

以下为各模块对外（public static）主要函数签名，供二次开发直接调用。

6.1 Admin（管理动作，返回输出文本）

函数	参数说明
doInit(baseDir, dn, bit, hash, algo)	初始化：DN/位数/哈希/算法
doMakeRoot(baseDir, dn, bit, hash, algo)	签发根证书
doList(baseDir)	列出登记表
doSignCert(baseDir, dn, bit, hash, type, years, sanType, sanContent, issuer, algo)	签发下级证书
doRevoke(baseDir, serial, reason)	吊销
doSignCrl(baseDir)	刷新 CRL
doInitOcsp(baseDir, algo)	签发 OCSP 证书（algo=RSA/ECC/SM2）
doInitTimestamp(baseDir, hash)	签发 TSA 证书（hash=sha1/sha256/sm3）
doVerifyCert(baseDir, target)	证书验证（文件或序列号）

6.2 TimestampUtil（时间戳）

函数	说明
parseRfc3161Request(der) → Rfc3161Request	解析 RFC3161 请求（hashAlgOid/messageImprint/nonce/certReq）
createRfc3161Response(reqDer, tsaCert, chain, signer, digestCalc) → byte[]	生成 RFC3161 响应 DER
createRfc3161ResponseStandard(reqDer, tsaCert, chain, key, sigAlg, digestAlg) → byte[]	RSA/ECC 便捷生成
createAuthenticodeTimestamp(content, cert, key, digitype) → byte[]	Authenticode PKCS#7（digitype 1/2/3/4=sha1/256/384/512）
SM3DigestCalculator	SM3 摘要计算器（SM2 TSA 用）

6.3 OcspUtil（OCSP）

函数	说明
parseRequest(der) → OcspRequest	解析请求（serialNumber/hashAlgOid/nonce）
createResponse(keyType, respCert, respKey, issuerCert, serial, status, reason, revokedAt, digestOid, nonce) → byte[]	生成签名响应（keyType=RSA/EC/SM2 自适应摘要）
unauthorizedResponse() → byte[]	5 字节无签名响应

6.4 Sm2Util（国密）

函数	说明
getSm2UidBytes() → byte[]	默认用户标识 1234567812345678
sm3(data) → byte[]	SM3 摘要 32 字节
computeZa(pub, uid) → byte[]	SM2 ZA 值
sm2Sign(priv, tbs) → byte[]	SM2 签名
sm2Verify(pub, tbs, sig) → boolean	SM2 验签
toEcPublicParams(pub) → ECPublicKeyParameters	JCA 公钥转 BC 参数

6.5 CertsDB / Config / MLog / PemIO

函数	说明
CertsDB.load(baseDir) / reload()	加载 / 重载 CERTS.txt
CertsDB.findBySerial(serial) → CertEntry	按序列号查（忽略大小写）
CertsDB.findIssuerIdBySerial(serial) → String	查颁发者序列号
CertsDB.getAll() → List	全部条目
Config(baseDir)	加载 CONFIG.txt
Config.getCpsWithCid(certId) → String	CPS URL 替换 {CID}
Config.getCpsNonice() → String	用户通告文本（\n→换行）
MLog.append(baseDir,op,cmd,result)	追加日志
MLog.appendErr(baseDir,op,cmd,error)	追加错误日志
PemIO.writeCert(path,cert) / writePrivateKey(path,key,type)	PEM 写出
PemIO.detectKeyType(path) → String	探测密钥类型
CertVerify.run(filename)	证书验证入口
RootGetCrl.run(baseDir)	生成全部 CRL

7. 配置文件与数据格式

7.1 CONFIG.txt（PKI/CONFIG.txt）

```
WEBPORT=80                          # WEB 服务监听端口
OCSP=http://localhost:80/OCSP        # 写入证书 AIA 的 OCSP URL
CRL=http://localhost:80/CRL/{CID}.crl   # CRL 分发点（{CID}=序列号占位符）
CRT=http://localhost:80/CRT/{CID}.crt   # calssuers URL
CPS=https://gitee.com/wxgshuju/modpkica # 证书策略 CPS URI
CPS_Nonice=http://example.com/nonice    # UserNotice 文本（多行以 \n 存储）
```

7.2 CERTS.txt（PKI/CERTS.txt）

```
# 序列号         类型         状态          状态原因编号    签发时间/吊销时间         颁发者ID         算法
#              ROOT/CA/END  V正常R吊销N未知    0-9     例如 2000/06/17-12:12:12
13fbe86586d1bca R V 0 2026/09/08-23:18:34 13fbe86586d1bca RSA
113521bfd       E R 5 2026/09/09-00:53:25 1cdd332012fa2   RSA
```

7.3 logs.log 记录格式（PKI/logs.log）

```
时间: 2026/09/12 01:29:47
时间戳: 1789147787217
操作: 时间戳响应
命令:
结果: 长度=3308字节


（每条记录后两个换行分隔）
```

8. WebAdmin 面板与 API

WebAdmin 内嵌于 MAIN，也可独立运行（modpkica.WebAdmin）。管理端口默认 8080，可用系统属性 -Dmodpkica.admin.port=端口 修改。深色主题，13 个页面。

8.1 页面一览

页面	关键功能
仪表盘	MAIN 状态、证书链树（R→C→E 悬停详情、R 展开/C 收起）、刷新全部 CRL、签发全部 OCSP、签发全部时间戳、最近日志
证书库	CERTS.txt 列表（筛选/刷新）、详情按钮弹出 JSON 结构化证书（X509JsonConverter）
签发根证书	DN/位数/哈希/算法
签发证书	DN（key=value 或 OID）、类型、密钥算法（RSA/ECC/SM2）、密钥位数联动、哈希、KeyUsage 下拉+悬浮多选、EKU 悬浮选择器（35 项）、有效期（年数或起止秒级）、SAN
吊销证书	序列号 + 原因
CRL 刷新 / OCSP 签发 / 时间戳签发	一键批量生成
证书验证	输入文件或序列号
初始化环境	清空重建登记表
系统配置	可视化读写 CONFIG.txt（CPS_Nonice 多行输入）
操作日志	时间范围（1/6/12/24 小时/全部）、行数、自动刷新 3s

8.2 API 一览（16 个，均 POST）

```
/api/status   服务状态      /api/certs     证书库列表     /api/logs      日志(按小时范围)
/api/config   配置读写      /api/chain     证书链树       /api/certjson  证书 JSON 详情
/api/version  版本          /api/issuers   颁发者下拉     /api/init      初始化环境
/api/makeroot 签发根        /api/signcert  签发证书       /api/revoke    吊销
/api/crl      刷新 CRL      /api/initocsp  签发 OCSP      /api/inittimestamp 签发 TSA
/api/verify   证书验证
```

9. 附录：构建与命令行速查

9.1 构建

```
Windows:  build.cmd            # 双击或 cmd 执行
Linux:    chmod +x build.sh && ./build.sh
等价命令: javac -encoding UTF-8 -cp "BC185\*" -d out src\modpkica\*.java
```

9.2 运行 MAIN（含 WebAdmin）

```
set MODPKICA_BASE=C:\Users\28299\Desktop\modpkica4JAVA
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.MAIN
# 管理面板 http://localhost:8080 ；WEB 服务 http://localhost:80
```

9.3 其他主程序示例

```
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.Admin
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.MakeRoot "CN=Root,O=My Org,C=CN" 2048 sha256 RSA
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.MakeCert "CN=www.test.com" 2048 sha256 1 1 0 1 "www.test.com" null null root RSA
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.RootGetCrl
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.CertVerify 113521bfd
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*" modpkica.Auto revoke 113521bfd 5
```

9.4 常见问题速查

现象	处理
中文乱码	运行加 -Dfile.encoding=UTF-8
端口占用 BindException	netstat -ano | findstr :80 结束旧进程，或改 WEBPORT
OCSP 未知序列号	确认 CERTS.txt 有记录 + WebPublic/CRT/序列号.crt 存在 + 已 initOCSP
Wosign RFC3161 失败	微软代码签名选 Authenticode 时间戳（/timestamp）
ECC 签发报错	ECC 位数只能 256/384/521；确认 BC185 完整

MODPKICA Java 复刻版 v1.0（BC 1.85 离线）— 开发助手文档 · 完